DU Starmap Navigator v2.3 — Setup & User Guide

Overview

The Navigator is a set of Programming Boards (PBs) for Dual Universe that let you save, share, and navigate to waypoints and routes. There are five PBs:

File	Purpose
Navigator_Ship_Screen	Ship PB with a clickable screen UI
Navigator_Ship_NoScreen	Ship PB with an AR HUD (no screen needed)
Navigator_Base	Personal base PB — stores your private waypoints
Navigator_OrgBase_Admin	Org base admin PB — manages org-shared waypoints
Navigator_OrgBase_Sync	Org base sync PB — serves waypoints to ships on request

Most players will use one ship PB and Navigator_Base. If you are part of an org, the org admin sets up the two OrgBase PBs at the org's base — you just need your ship PB.

Hardware You Need

Ship (Screen version)

- 1x Programming Board
- 1x Screen Unit (any size)
- 1x Databank
- 1x Receiver
- 1x Emitter
- 1x Databank *(optional — only needed for Arch HUD or Saga HUD integration)*

Ship (No Screen version)

- 1x Programming Board
- 1x Databank
- 1x Receiver
- 1x Emitter
- 1x Screen Unit *(optional — only needed for the Theme Editor)*
- 1x Databank *(optional — only needed for Arch HUD or Saga HUD integration)*

Personal Base

- 1x Programming Board
- 1x Screen Unit
- 1x Databank
- 1x Receiver
- 1x Emitter

Org Base (if running an org)

You need TWO programming boards at the base:

- **Admin PB**: Screen + Databank
- **Sync PB**: Screen + Databank (shared with Admin) + Receiver + Emitter

---

Slot Connections

All five PBs use **slot auto-detection** — link the elements they need to any of the PB's slots, in any order. At startup, each script probes every linked element and figures out what it is; there's no fixed slot-number table to follow.

Navigator_Ship_Screen needs: Screen Unit, Databank, Receiver, Emitter, plus an optional 2nd Databank (shared with Arch HUD or Saga HUD, for HUD integration).

Navigator_Ship_NoScreen needs: Databank, Receiver, Emitter, plus an optional Screen Unit (for the mouse-driven Theme Editor only) and an optional 2nd Databank (same Arch/Saga HUD integration purpose).

Navigator_Base needs: Screen Unit, Databank, Receiver, Emitter.

Navigator_OrgBase_Admin needs: Screen Unit, Databank (the same physical databank as the Sync PB — see below). No Receiver/Emitter — the Admin PB doesn't talk to ships directly, that's the Sync PB's job.

Navigator_OrgBase_Sync needs: Screen Unit, Databank (the same physical databank as the Admin PB), Receiver, Emitter.

After activating, check the Lua console — it prints a confirmation line such as:

```
[NAV] slot1=databank  slot2=receiver  slot3=emitter  slot5=navdatabank
```

(`[BASE]` on Navigator_Base, `[ORG-ADMIN]` on Navigator_OrgBase_Admin, `[ORG-SYNC]` on Navigator_OrgBase_Sync.) Only linked slots are listed. Each element shows the role it was detected as (`databank`, `navdatabank`, `receiver`, `emitter`, `screen`), or `unrecognized` if it couldn't identify what's linked there — in which case double-check that slot actually holds one of the supported element types.

How the two databanks are told apart (ship PBs only): if only one Databank is linked, it's used as the primary. If two are linked:

1. **Naming hint (most reliable):** if you've renamed the Arch/Saga databank in-game (right-click the element → Rename) to include "arch", "saga", "hud", or "nav2" anywhere in the name, it's recognized as `navdatabank` regardless of what's in it. Recommended, especially for a brand-new setup where both databanks start out empty. Confirmed working in-game 2026-09-06.
2. **Contents:** otherwise, whichever one already has Navigator's own data in it (waypoints, theme, etc.) is treated as primary and the other as `navdatabank`.
3. **Slot order (last resort):** if neither hint applies, it falls back to slot order (first found = primary) — and tells you it did, both in the Lua console **and** on the ship's own display (AR HUD status line or screen footer, whichever this PB has), so you don't have to be watching Lua chat to notice:
    - Both databanks completely blank and unnamed → `Check console: DB guess` on the HUD/screen, full detail in the console.
    - One of them has *some* data, just not Navigator's or Arch/Saga's (e.g. a databank reused from another script) → `Check console: DB link?` on the HUD/screen — this is the stronger warning, since it suggests the wrong element may be linked rather than just an ambiguous fresh install.

The console line shows each linked element's detected name in parentheses when available, e.g. `slot5=navdatabank(ArchDB)` — use it to confirm it picked correctly, and if not, just swap which physical databank goes to your Arch/Saga seat (or rename it).

> **Important:** The Admin and Sync PBs must share the **same physical databank**. The Admin writes data, the Sync reads and serves it to ships. This still works with auto-detect — it's the same physical element being linked to both PBs, regardless of which slot number it lands in on each.

Export Parameters (Right-click PB → Edit LUA Parameters)

Ship PBs (Screen and NoScreen)

Parameter	Default	Description
`CustomAtlas`	`atlas`	Atlas file to load. Leave as default unless you have a custom atlas in `autoconf/custom/`.
`BaseChannel`	`NavBase`	Channel name for your personal base. Must match your base PB's channel.
`CalcSpeed`	`0`	Your cruise speed in space in km/h for travel time calculations. `0` auto-detects your ship's actual max speed (`construct.getMaxSpeed()`) every time the PB loads. Set a specific value to plan at a slower speed instead — e.g. to save fuel, or to see how long a trip takes if you're running low and can't push max speed.
`CalcThrust`	`0`	Your ship's total thrust in kN — read this directly from your ship's stats screen. Acceleration is calculated automatically using your ship's current mass. Leave at `0` to use `CalcAccel` instead.
`CalcBrake`	`0`	Your ship's total brake force in kN from your ship's stats screen. When set, the script decelerates using brakes rather than a flip-and-burn. Leave at `0` to let the script read brake force automatically from the construct; if that also fails it falls back to using thrust for deceleration.

Parameter	Default	Description
`CalcAccel`	`5`	Fallback acceleration in m/s² — only used if `CalcThrust` is `0`.
`AccentR/G/B`	`0 / 200 / 255`	Starting accent color (RGB 0–255). The in-game Theme Editor overrides this once you save a theme.

No Screen version only

Parameter	Default	Description
`HudX`	`13`	HUD left position as a percentage from the left screen edge. Change this and reimport to move the HUD without needing to be seated.
`HudY`	`15`	HUD top position as a percentage from the top screen edge. Same as above.

> You can also move the HUD while seated using the `hudpos X Y` chat command — changes take effect immediately without reimporting.

Navigator_Base

Parameter	Default	Description
`BaseChannel`	`NavBase`	Channel ships use to reach this base. Must match the ships' `BaseChannel`.

Navigator_OrgBase_Admin

Parameter	Default	Description
`OrgChannel`	`NavOrg`	Channel for this org. Ships use this when doing their first org sync.
`OrgName`	`MyOrg`	Display name shown on the screen and sent to ships during sync.

Navigator_OrgBase_Sync

No export parameters. It reads its channel and org name from the shared databank written by the Admin PB on first start.

Personal Setup (Most Players)

1. Place Navigator_Base at your base. Connect Screen, Databank, Receiver, Emitter in slot order.
2. Place your ship PB on your ship. Connect elements in slot order.
3. Set `BaseChannel` to the same value on both (the default `NavBase` works if you only have one base).
4. Turn on the base PB — it shows a screen with tabs.
5. Turn on the ship PB — it loads your waypoints from the databank.
6. Type `sync` in Lua chat on the ship to pull waypoints from the base.

Sync Whitelist (Personal Base)

By default the base PB responds to a sync request from any ship that knows the channel name. If you want to restrict which ships can sync, you can maintain a whitelist on the base.

The whitelist is empty by default — empty means allow all. You only need to set it up if you are worried about others syncing to your base.

Whitelist commands (type in Lua chat while the base PB is running)

Command	Description
`allow NAME`	Add a player name or ship ID to the whitelist
`deny NAME`	Remove an entry from the whitelist
`allowlist`	Show the current whitelist

By player name vs ship ID

- `allow ToobsTube` — allows **all ships** flown by that player. Best for whitelisting yourself or a trusted friend regardless of which ship they are in.
- `allow MyShip#abc123` — allows only that **specific ship** regardless of who is flying it.

You can mix both types in the same list.

Finding the right name to allow

Two ways:

1. **Let it tell you:** If the whitelist is non-empty and a ship is denied, the Lua console on the base prints: `[BASE] Sync denied: MyShip#abc123 (ToobsTube)` Copy the player name or ship ID from that line and use `allow` with it.

2. **From a successful sync:** The base status line shows `Sending N items to: MyShip#abc123 (ToobsTube)` when a sync is accepted. Use whatever name you want to whitelist.

Example — locking down your base to your own ships

```
allow ToobsTube
allowlist
```

That one entry covers every ship you fly, now and in the future.

Org Setup

At the Org Base

1. Place both the Admin PB and the Sync PB.
2. Connect both to the **same databank**.
3. Connect the Admin PB to its own Screen only.
4. Connect the Sync PB to its own Screen, Receiver, and Emitter.
5. Set `OrgChannel` on the Admin PB (e.g. `NavAlliance`).
6. Set `OrgName` on the Admin PB (e.g. `Alliance`).
7. Turn on the Admin PB first — this writes the channel and name to the databank.
8. Turn on the Sync PB — it reads those values automatically. The channel name is shown on its screen.

On Each Ship (First-time connection to an org)

1. Make sure your ship PB is running and your Receiver is in range of the org's Sync PB.
2. Type `firstsync CHANNEL` in Lua chat — use the channel shown on the org's Sync PB screen.
    - Example: `firstsync NavAlliance`
3. The ship syncs with that org and creates a new tab for it automatically. The org channel is saved — you never need to type it again.
4. After that, select the org's tab and use **SETTINGS** → **SYNC FROM ORG**, or type `orgsync`, to refresh org waypoints.

For a second org, just repeat: type `firstsync CHANNEL` with the second org's channel. Each org gets its own tab.

Using the Screen Version

The screen has three panels:

- **Left — Waypoints:** Click a waypoint to select it.
- **Middle — Routes/Stops:** Click a route to open its stops. The route's name shows on the row under the title bar; click the **ROUTES** title bar to go back to the route list.
- **Right — Navigation + Buttons:** Shows your current nav target, distance, travel time, and action buttons. Click **SETTINGS** in the footer to swap this panel for the settings panel.

Long lists show a scrollbar on the right edge: click the arrows to step one row at a time, or press and drag on the bar to jump.

Tabs

- **Personal** — your private waypoints
- **[Org name]** — org waypoints (appear after first org sync)
- **ATLAS** — all bodies in the game. The list uses both the left and middle panels (two columns). Click a body, then **NAVIGATE TO BODY**.

Navigation panel buttons

Button	What it does
MARK WP	Saves your current ship position as a new waypoint with an auto-generated name
MARK STOP	Adds your current position as the last stop on the selected route
NEW ROUTE	Prompts you in chat to type a name, then creates an empty route
ADD STOP	Starts adding stops to the selected route: click waypoints in the left panel to add them in order (or type a WP name / `::pos{}` in chat). The button changes to **DONE ADDING** — click it when finished
RENAME	Prompts you in chat to type a new name for the selected WP, route, or stop
SET COORDS	Prompts you in chat to paste `::pos{}` coords for the selected WP or stop
NAV WP	Sets the selected waypoint as your nav target
NAV ROUTE	Starts the selected route from stop 1
NAVIGATE TO BODY	*(ATLAS tab)* Sets the selected planet/moon as your nav target
NEXT STOP	Advance to the next route stop
AUTO FLY	Toggle automatic route flying — Navigator advances stops and engages autopilot automatically. Shows ON / OFF
CLEAR NAV	Removes your current nav target
PRINT COORDS	Prints the selected waypoint's coordinates to Lua chat as a clickable nav marker
LK *(in waypoint list)*	Toggle waypoint lock — locked WPs show an LK badge and are excluded from push/sync

Settings panel

Click **SETTINGS** in the footer. The panel shows the version, ship ID, base channel, and the channel of the org tab you're on. Click **BACK** (or **CLOSE** in the footer) to return.

Button	What it does
SYNC FROM BASE	Pulls waypoints/routes from your personal base
PUSH TO BASE	Sends your waypoints/routes to your personal base
SYNC FROM ORG	Pulls waypoints from the org on the active tab *(org tab only)*
PUSH TO ORG	Sends waypoints to the org on the active tab *(org tab only)*
FIRST ORG SYNC	Shows a hint — type `firstsync CHANNEL` in Lua chat to add a new org
THEME	Opens the Theme Editor color picker
BACK	Returns to the navigation panel

Map marker

When you navigate to a waypoint, route stop, or body, Navigator sets it as your DU map destination once. After that it leaves the map alone, so you can set your own destination at any time without clearing nav. Click NAV / NEXT STOP again to put Navigator's target back on the map.

Guided Chat Buttons

NEW ROUTE, ADD STOP, RENAME, and SET COORDS work by prompting you in chat. When you click one, a status message appears telling you what to type. Your next message in Lua chat is consumed as the input — normal commands are bypassed until you respond (or until the PB restarts if you want to cancel).

Example — rename a waypoint:

1. Click the waypoint in the left panel to select it.
2. Click **RENAME**.
3. Status bar shows `Type new WP name in chat`.
4. Type the new name in Lua chat and press Enter.

Chat Commands (screen version)

Type these in Lua chat while the PB is running. Most editing actions can also be triggered by the on-screen buttons, which prompt you for input rather than requiring you to remember command syntax.

Command	Description
`add NAME ::pos{...}`	Add or update a waypoint
`add NAME`	Add a waypoint at your current position
`rename NEWNAME`	Rename the selected WP, route, or stop
`setpos ::pos{...}`	Update coords of the selected WP or stop
`del`	Delete the selected WP, route, or stop
`newroute NAME`	Create a new route
`addstop WAYPOINT NAME`	Add a saved waypoint as the last stop on the selected route
`addstop ::pos{...}`	Add coordinates as the last stop on the selected route
`addstop NAME; NAME; ...`	Add several stops at once, in that order (e.g. `addstop Home; Mine; ::pos{...}`)
`delstop N`	Remove stop number N from the selected route
`lock WPNAME`	Lock a waypoint — prevents it being pushed to base or overwritten by sync
`unlock WPNAME`	Remove the lock from a waypoint
`sync`	Sync from personal base
`orgsync`	Sync from the org on the current tab
`firstsync CHANNEL`	First-time sync with a new org (e.g. `firstsync NavOrg`)

Command	Description
`push`	Push to personal base
`orgpush`	Push to the org on the current tab
`importarch`	Import all Arch HUD SavedLocations as personal WPs
`importsaga`	Import all SAGA routes — single-stop as WPs, multi-stop as routes
`navdbkeys`	Print all navdatabank key names to Lua chat (diagnostic)
`theme ...`	Theme commands (`show`, `save`, `load`, `push`, `export`, `import`, etc.) — see THEME_GUIDE.md
`nhelp`	Show all commands in Lua chat (`navhelp` also works)

> **Why `nhelp`?** Every running script receives every Lua chat line. Flight scripts use `help` too, so Navigator's ship PBs answer `nhelp` instead and leave `help` to your flight script. Navigator also stays quiet about common flight-script commands (`ap`, `goto`, `hud`, `stop`…) instead of replying "Unknown". Base PBs still use `help`.

Using the Base / Org Admin Screen

The base and org admin screens share the same tab layout:

- **WAYPOINTS tab** — lists all waypoints. Click one to select.
- **ROUTES tab** — lists all routes. Click one to open its stops view.
- **PENDING tab** *(Org Admin only)* — shows waypoints submitted by members waiting for approval.

Action bar buttons

The row of buttons at the bottom changes depending on the current view:

Waypoints view:

Button	What it does
ADD WP	Prompts in chat to type a name, saves your current position as a waypoint
RENAME	Prompts in chat — renames the selected waypoint
SET COORDS	Prompts in chat — paste `::pos{}` to update the selected waypoint's coordinates
PRINT	Prints the selected waypoint and its `::pos{}` coords to Lua chat as a clickable nav marker
DELETE	Deletes the selected waypoint

Routes view:

Button	What it does
NEW ROUTE	Prompts in chat to type a name and creates an empty route
RENAME	Prompts in chat — renames the selected route
PRINT	Prints the selected route's stops to Lua chat
DELETE	Deletes the selected route

Stops view *(after clicking a route)*:

Button	What it does
ADD STOP	Prompts in chat to type a WP name or `::pos{}`, adds it to the end of the route
RENAME	Prompts in chat — renames the selected stop
SET COORDS	Prompts in chat — paste `::pos{}` to update the selected stop's coordinates
PRINT	Prints the stop's coordinates to Lua chat as a clickable nav marker
DEL STOP	Deletes the selected stop

Click the **ROUTES** tab while in the stops view to return to the route list.

Using the No Screen Version

The HUD appears as an AR overlay. Press **Left Shift** to show or hide it.

AR HUD Positioning

The HUD position is set by the `HudX` and `HudY` export parameters (percentage from the left and top edges of the screen). You can change these and reimport the PB to move the HUD, or use the chat command while seated:

- `hudpos X Y` — move the HUD immediately, e.g. `hudpos 10 20`
- `hudpos` — print the current HUD position to Lua chat

Navigation Controls

Keys	Action
Alt + Up / Down	Move between sections (left panel) or items (right panel)
Alt + Right	Enter the right panel / activate selected item
Alt + Left	Go back to the left panel
Alt + 0	Open / close the Theme Editor (requires an optional Screen Unit linked)
Left Shift	Toggle HUD on/off

Sections

Section	Contents
WP	Your personal waypoints. Each waypoint shows its `::pos{}` coords below it (dimmed). Select the waypoint name and press Alt+Right to navigate.
ORG	Org waypoints and routes (after syncing).
ROUTES	Your personal routes. Each route lists its stops as sub-items with distance from current position. Select a stop name and press Alt+Right to navigate directly to that stop.
SETTINGS	Mark WP, Next/Prev Stop, Clear Nav, Sync, Push actions.
ATLAS	All game bodies. Select one and press Alt+Right to navigate to it.
TIME CALC	Travel time to all your waypoints from current position.

Chat Commands (no screen version)

Command	Description
`add NAME`	Add a waypoint at your current position
`add NAME ::pos{...}`	Add or update a waypoint with specific coords
`del NAME`	Delete a waypoint by name
`newroute NAME`	Create a new route
`addstop ROUTE > WAYPOINT NAME`	Add a saved waypoint as the last stop on a route
`addstop ROUTE > ::pos{...}`	Add coordinates as the last stop on a route
`addstop ROUTE > NAME; NAME; ...`	Add several stops at once, in that order
`addstop ROUTE > here`	Add your current nav target as the last stop
`delstop ROUTE N`	Remove stop number N from a route
`delroute NAME`	Delete a route
`nav NAME`	Navigate to a waypoint or route by name
`nav off`	Clear navigation
`next` / `prev`	Next / previous route stop
`autofly on` / `autofly off`	Toggle Auto Fly — auto-advances stops and engages HUD autopilot
`lock WPNAME`	Lock a waypoint — prevents push to base or overwrite by sync
`unlock WPNAME`	Remove the lock from a waypoint
`coords WPNAME`	Print a waypoint's coordinates to the Lua console for copying
`sync`	Sync from personal base

Command	Description
`orgsync`	Sync from the active org
`firstsync CHANNEL`	First-time sync with a new org (e.g. `firstsync NavOrg`)
`push`	Push to personal base
`orgpush`	Push to the active org
`org NAME`	Switch active org context
`search NAME`	Filter atlas by name
`search`	Clear atlas filter
`hudpos X Y`	Move the HUD while seated (e.g. `hudpos 10 20`)
`hudpos`	Show current HUD position
`status`	Show current nav target and distance
`list`	List all waypoints
`routes`	List all routes
`importarch`	Import all Arch HUD SavedLocations as personal WPs
`importsaga`	Import all SAGA routes — single-stop as WPs, multi-stop as routes
`navdbkeys`	Print all navdatabank key names to Lua chat (diagnostic)
`theme ...`	Theme commands (`show`, `save`, `load`, `push`, `export`, `import`, etc.) — see THEME_GUIDE.md
`nhelp`	Show all commands in Lua chat (`navhelp` also works)

The `>` separates the route name from the stops, so route names with spaces work (`addstop Ore Run > Home; Mine`). The older `addstop ROUTE WP` form still works for one-word route names.

Arch HUD Integration

The Navigator can send waypoints directly to Arch HUD as a temporary navigation target. No receiver or extra hardware needed — it uses a shared databank.

Setup

1. In the release ZIP the file is already placed at the correct path: `autoconf\custom\archhud\archhud_userclass.lua` Copy that folder into your game's `data\lua\` directory, then **rename** `archhud_userclass.lua` **to** `userclass.lua` in that folder. It will not work until it is renamed. The file is left with its original name so it does not overwrite an existing `userclass.lua` if you already have one — see the note in the file itself if you need to merge.

2. In the game, link the **same databank** that your Arch HUD control seat uses to any open slot on your Navigator PB (it's auto-detected — see Slot Connections above).

3. That's it. When the Navigator PB starts it will print `[NAV] HUD bank=OK (Arch+Saga)` confirming the connection.

How it works

Every time you navigate to a waypoint or route stop, the Navigator writes the destination to the shared databank. Arch HUD reads it on the next tick, sets it as a temporary nav target (same as typing the `::pos` in chat), and clears the key. Nothing is saved permanently in Arch.

You will see these messages in Lua chat when it fires:

- `[NAV] HUD bank=OK (Arch+Saga)` — on PB start, confirms the databank is linked
- `[NAV] sent to HUD bank: NAME` — each time a waypoint is sent

Saga HUD Integration

The Navigator can send waypoints directly to Saga HUD 4.22 as a temporary navigation target using the same shared databank as Arch HUD.

Setup

1. In the release ZIP you will find `Saga_AP_4.22_Nav.json`. Import this into your Saga control seat instead of the standard Saga JSON.
2. Link the **same databank** that your Navigator PB detected as its `navdatabank` (check the startup console line) to your Saga control seat as well.
3. That's it. When the Navigator PB starts it will print `[NAV] HUD bank=OK (Arch+Saga)` confirming the connection.

How it works

Every time you navigate to a waypoint or route stop, the Navigator writes the destination to the shared databank. Saga reads it on the next tick, calls its internal `/goto` equivalent to set a temporary nav target, and clears the key. Nothing is saved permanently in Saga's route database.

If **Auto Fly** is enabled in Navigator, Saga's autopilot is also engaged automatically after the target is set.

You will see these messages in Lua chat:

- `[NAV] HUD bank=OK (Arch+Saga)` — on PB start
- `[NAV] sent to HUD bank: NAME` — each time a waypoint is sent
- `[NAV] Target: NAME` — from Saga confirming it received the target

Importing from Arch HUD or SAGA

If you're already using Arch HUD or SAGA and have saved locations/routes, you can import them directly into the Navigator with one command. No manual re-entry needed.

Requirements

Both HUDs must share a databank with the Navigator PB (the `navdatabank` slot). If you already have Arch or SAGA integration set up, this databank is already linked — just run the command.

Importing from Arch HUD

Arch stores saved locations in a key called `SavedLocations` in its databank.

```
importarch
```

All saved locations come in as personal waypoints, using the name and world coordinates from Arch.

Importing from SAGA

SAGA stores routes in a key called `SagaRoutes` in its databank.

```
importsaga
```

- **Single-stop routes** → imported as personal **waypoints**
- **Multi-stop routes** → imported as personal **routes** with all stops and stop names preserved

Safe to re-run

Both commands are safe to run multiple times. Existing entries with matching names are updated rather than duplicated.

Note on switching between Arch and Saga

The same databank works for both HUDs — no link changes needed when switching. The Navigator writes both `nav_arch_dest` and `nav_saga_dest` keys every time. Whichever HUD is running will pick up its own key and ignore the other.

Theme Editor

All PBs include a built-in color picker to customize the UI colors.

- **Screen version:** Click the **THEME** button in the navigation panel.
- **No screen version:** Press **Alt+0** (requires an optional Screen Unit linked — any slot, auto-detected).
- **Base and Org PBs:** Click the **THEME** button on the screen.

The picker shows 8 color slots. Select a slot on the left, then drag the sliders to adjust hue, saturation, and brightness. The right panel shows a split preview — the left half shows the saved color, the right half shows your current change live. Click **SAVE** to apply. Click **RESET** to go back to the saved color. Themes are stored in the databank and survive PB restarts.

See THEME_GUIDE.md for a full description of what each color slot controls.

Travel Time Calculator

Set these export parameters on your ship PB:

- **CalcSpeed** — your cruise speed in space in km/h. Leave at `0` (default) to auto-detect your ship's actual max speed automatically — the TIME CALC screen/panel shows `(auto)` next to the speed when this is happening. Set a specific value instead to plan at a slower speed on purpose, e.g. to save fuel on a long trip or to check timing when you're running low and can't push max speed.
- **CalcThrust** — your ship's total thrust in kN, shown on the ship stats screen. The script divides this by your current mass automatically, so the calculation stays accurate as your cargo changes. Recommended over `CalcAccel`.
- **CalcBrake** — your ship's total brake force in kN, also shown on the ship stats screen. When provided, the deceleration phase uses brakes rather than a flip-and-burn, giving a more accurate time estimate. If left at `0` the script tries to read brake force directly from the construct automatically. If that fails too it falls back to using thrust for deceleration.
- **CalcAccel** — fallback acceleration in m/s². Only used if `CalcThrust` is left at `0`.

The calculator uses real burn physics: acceleration burn at the start, cruise at top speed, then deceleration. If the destination is too close to reach top speed, it calculates a triangle burn profile instead.

Travel time shows:

- **Screen version:** Next to the distance in the Navigation panel (e.g. `257 su ▸ 5h 23m`)
- **No screen version:** In the TIME CALC section, listed for every waypoint

---

Utility Tools

The `dist/tools/` folder contains standalone programming board scripts for maintenance tasks. Import each `.txt` file into a separate PB.

Databank_Copy

Backs up or restores a databank by copying all keys to another databank. Useful before testing imports or making bulk changes.

Setup: Link two databanks to any slots on the PB. The tool auto-detects them.

Commands:

Command	Description
`copy`	Copy DB1 → DB2 (backup)
`restore`	Copy DB2 → DB1 (restore)
`yes`	Confirm overwrite if destination already has data
`cancel`	Cancel a pending overwrite

> **Tip:** To get an empty backup databank, either use the Wipe_Databanks tool first, or take the databank into your inventory and right-click → Clear.

Databank_Inspector

Displays all keys and values stored in a linked databank on a screen. Useful for diagnosing what data a HUD is storing.

Setup: Link a screen and the databank you want to inspect to any of the PB's slots, in any order — auto-detected at startup, same as the main PBs. Check the Lua console for a `[INSP] slot1=...` line confirming what was detected.

Commands: Type a key name in Lua chat to filter. Type `clear` to reset. Type `next` / `prev` to page through results.

Wipe_Databanks

Clears all data from any number of linked databanks. Link one or more databanks to any of the PB's slots — auto-detected, no fixed slot names needed. Activating the PB wipes all of them immediately — use with care.

Channel Setup Tips

- Channels are text strings — they must match exactly between sender and receiver.
- Make channel names unique to avoid picking up traffic from other players' navigators nearby.
- The org channel is shown on the Sync PB's screen — that's what you type in `firstsync`.
- Emitters and Receivers must be within range of each other (same construct is always in range).

Troubleshooting

Sync doesn't work / no waypoints appear

- Check that `BaseChannel` matches exactly on both the ship and the base PB.
- Make sure the base PB is running before you type `sync`.
- Make sure Receiver and Emitter are linked to the correct PBs.

Base Lua console says "Sync denied"

- Your base has a sync whitelist set up and this ship/player is not on it.
- On the base PB, type `allowlist` to see who is allowed, then `allow PLAYERNAME` to add yourself.
- See the **Sync Whitelist** section above.

First org sync doesn't work

- Make sure the org's Sync PB is running and you can see its channel on the screen.
- Type `firstsync CHANNEL` exactly as shown — the channel is case-sensitive if you type it manually, but the script will still try to match it against known channels.
- Your ship's Emitter must be in range of the org Sync PB's Receiver.

Screen clicks don't work

- Make sure you imported the correct `.txt` file into the PB (not copy-pasted into the screen).
- Check the startup console line — the Screen Unit should show up as `screen`, not `unrecognized` or missing.

"No emitter" error

- The Emitter element isn't linked to the PB.
- Check the startup console line to confirm something was actually detected as `emitter`.

Waypoints show [0] on screen

- The databank may be empty — try typing `sync` first to pull from the base.
- If it's a fresh install with no base yet, type `add NAME ::pos{...}` to create your first waypoint.

Travel time shows ---

- You are not inside a construct with a core (no position available).
- Or the waypoint uses planet-relative coords that the atlas can't resolve.

Theme Editor doesn't open on No Screen version

- A Screen Unit must be linked (any slot — check the startup console line shows `screen` on one of them).
- The screen must be activated — it will show "THEME EDITOR" when the PB is running and the picker is closed.

Script error on start, HUD loads but responds slowly (No Screen version)

- The startup console line will show `unrecognized` for a linked element that isn't a Screen Unit, Databank, Receiver, or Emitter — that's almost always the culprit, since the script only knows how to handle those four types.
- If you have exactly the elements you expect linked and this still happens, check for a **second Databank auto-detection mismatch**: with two databanks linked and both still empty (fresh install), the script guesses which is primary by slot order — see "How the two databanks are told apart" in the Slot Connections section above. Swap which physical databank goes to your Arch/Saga seat if it guessed wrong.

Arch HUD does not respond to waypoints

- Check that `[NAV] HUD bank=OK (Arch+Saga)` prints when the Navigator PB starts. If it doesn't, no second databank was detected as `navdatabank` — check the startup console line.
- Make sure the `userclass.lua` file is placed at exactly: `Game\data\lua\autoconf\custom\archhud\userclass.lua`
- Make sure the databank detected as `navdatabank` is the same physical databank that your Arch HUD control seat is linked to.

Saga HUD does not respond to waypoints

- Make sure you imported `Saga_AP_4.22_Nav.json` (the patched version), not the standard Saga JSON.
- Check that `[NAV] HUD bank=OK (Arch+Saga)` prints when the Navigator PB starts.
- Make sure the databank detected as `navdatabank` (check the startup console line) is also linked to your Saga control seat.
- If you wiped the databank, Navigator's waypoints are also cleared — add a waypoint first before testing.
- You should see `[NAV] Target: NAME` in the Lua console from Saga when a target is received.

Saga autopilot enters parking mode and misses the destination (no braking)

- You are likely using an older version of the patched Saga JSON where `setTarget` was called before `resetAP`, leaving the AP with no destination and defaulting to parking mode.
- Fix: re-import `Saga_AP_4.22_Nav.json` from the current release ZIP — the corrected version is included.